

Data Preprocessing Code (Weather Dataset)

```
In [15]: import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
```

Load Dataset

```
In [2]: df = pd.read_csv("GlobalWeatherRepository.csv")
```

```
In [3]: print("Original Shape:", df.shape)
```

Original Shape: (123356, 41)

```
In [4]: df.head()
```

Out[4]:

	country	location_name	latitude	longitude	timezone	last_updated_epoch	las
0	Afghanistan	Kabul	34.52	69.18	Asia/Kabul	1715849100	2
1	Albania	Tirana	41.33	19.82	Europe/Tirane	1715849100	2
2	Algeria	Algiers	36.76	3.05	Africa/Algiers	1715849100	2
3	Andorra	Andorra La Vella	42.50	1.52	Europe/Andorra	1715849100	2
4	Angola	Luanda	-8.84	13.23	Africa/Luanda	1715849100	2

5 rows × 41 columns



Basic Info

```
In [5]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 123356 entries, 0 to 123355
Data columns (total 41 columns):
 #   Column                                  Non-Null Count  Dtype
---  -
 0   country                                123356 non-null  object
 1   location_name                          123356 non-null  object
 2   latitude                               123356 non-null  float64
 3   longitude                              123356 non-null  float64
 4   timezone                               123356 non-null  object
 5   last_updated_epoch                    123356 non-null  int64
 6   last_updated                           123356 non-null  object
 7   temperature_celsius                   123356 non-null  float64
 8   temperature_fahrenheit                 123356 non-null  float64
 9   condition_text                         123356 non-null  object
10   wind_mph                              123356 non-null  float64
11   wind_kph                              123356 non-null  float64
12   wind_degree                           123356 non-null  int64
13   wind_direction                         123356 non-null  object
14   pressure_mb                           123356 non-null  int64
15   pressure_in                           123356 non-null  float64
16   precip_mm                             123356 non-null  float64
17   precip_in                             123356 non-null  float64
18   humidity                              123356 non-null  int64
19   cloud                                 123356 non-null  int64
20   feels_like_celsius                     123356 non-null  float64
21   feels_like_fahrenheit                  123356 non-null  float64
22   visibility_km                          123356 non-null  float64
23   visibility_miles                       123356 non-null  int64
24   uv_index                              123356 non-null  float64
25   gust_mph                              123356 non-null  float64
26   gust_kph                              123356 non-null  float64
27   air_quality_Carbon_Monoxide            123356 non-null  float64
28   air_quality_Ozone                      123356 non-null  float64
29   air_quality_Nitrogen_dioxide           123356 non-null  float64
30   air_quality_Sulphur_dioxide            123356 non-null  float64
31   air_quality_PM2.5                      123356 non-null  float64
32   air_quality_PM10                       123356 non-null  float64
33   air_quality_us-epa-index                123356 non-null  int64
34   air_quality_gb-defra-index              123356 non-null  int64
35   sunrise                                123356 non-null  object
36   sunset                                  123356 non-null  object
37   moonrise                               123356 non-null  object
38   moonset                                123356 non-null  object
39   moon_phase                             123356 non-null  object
40   moon_illumination                      123356 non-null  int64
dtypes: float64(21), int64(9), object(11)
memory usage: 38.6+ MB

```

```
In [32]: df.describe(include="all")
```

Out[32]:

	country	location_name	latitude	longitude	timezone	last_updated_epo
count	122576	121947	1.225760e+05	1.225760e+05	0.0	1.225760e+
unique	211	255	NaN	NaN	NaN	N
top	bulgaria	kabul	NaN	NaN	NaN	N
freq	1441	630	NaN	NaN	NaN	N
mean	NaN	NaN	-1.368034e-17	-2.643319e-17	NaN	-2.493067e-
std	NaN	NaN	1.000004e+00	1.000004e+00	NaN	1.000004e+
min	NaN	NaN	-2.476313e+00	-1.899180e+00	NaN	-1.733525e+
25%	NaN	NaN	-6.321076e-01	-4.702074e-01	NaN	-8.657653e-
50%	NaN	NaN	-7.945998e-02	3.008797e-02	NaN	1.993976e-
75%	NaN	NaN	8.682283e-01	4.824407e-01	NaN	8.638042e-
max	NaN	NaN	1.840479e+00	1.911413e+00	NaN	1.731106e+

11 rows × 43 columns



In [33]:

```
df.isnull().sum()
```

```

Out[33]: country                0
location_name                629
latitude                    0
longitude                   0
timezone                   122576
last_updated_epoch          0
last_updated                0
temperature_celsius         0
temperature_fahrenheit      0
condition_text              0
wind_mph                    0
wind_kph                    0
wind_degree                 0
wind_direction              0
pressure_mb                 0
pressure_in                 0
precip_mm                   0
precip_in                   0
humidity                    0
cloud                       0
feels_like_celsius          0
feels_like_fahrenheit       0
visibility_km               0
visibility_miles            0
uv_index                    0
gust_mph                    0
gust_kph                    0
air_quality_Carbon_Monoxide 0
air_quality_Ozone           0
air_quality_Nitrogen_dioxide 0
air_quality_Sulphur_dioxide 0
air_quality_PM2.5           0
air_quality_PM10            0
air_quality_us-epa-index    0
air_quality_gb-defra-index  0
sunrise                     0
sunset                      0
moonrise                    0
moonset                     0
moon_phase                  0
moon_illumination           0
wind_kmh                    0
year_month                   122576
dtype: int64

```

```
In [6]: df.fillna(df.mean(numeric_only=True), inplace=True)
```

Handle Missing Values

```
In [23]: num_cols = df.select_dtypes(include=np.number).columns
df[num_cols] = df[num_cols].fillna(df[num_cols].median())
```

```
In [24]: cat_cols = df.select_dtypes(include="object").columns
for col in cat_cols:
```

```
df[col]=df[col].fillna(df[col].mode()[0])
```

Remove Duplicates

```
In [25]: df.duplicated().sum()
```

```
Out[25]: np.int64(0)
```

```
In [36]: df.drop_duplicates(inplace=True)
```

Unit Conversion Example

```
In [27]: if "temperature_f" in df.columns:  
         df["temperature_c"] = (df["temperature_f"] - 32) * 5/9
```

```
In [28]: if "wind_mph" in df.columns:  
         df["wind_kmh"] = df["wind_mph"] * 1.60934
```

Convert Date Columns

```
In [41]: date_cols = [c for c in df.columns if "date" in c.lower()]  
         for col in date_cols:  
             df[col]=pd.to_datetime(df[col], errors="coerce")
```

Fix Inconsistent Text Data

```
In [45]: for col in cat_cols:  
         df[col]=df[col].astype(str).str.strip().str.lower()
```

Outlier Handling

```
In [42]: for col in num_cols:  
         Q1 = df[col].quantile(0.25)  
         Q3 = df[col].quantile(0.75)  
         IQR = Q3 - Q1  
         lower = Q1 - 1.5 * IQR  
         upper = Q3 + 1.5 * IQR  
         df[col] = np.where(df[col] < lower, lower,  
                             np.where(df[col] > upper, upper, df[col]))
```

Feature Scaling

```
In [30]: scaler = StandardScaler()  
         df[num_cols] = scaler.fit_transform(df[num_cols])
```

Aggregation Example

```
In [50]: import pandas as pd

if len(date_cols) > 0:
    date_col = date_cols[0]
    df[date_col] = pd.to_datetime(df[date_col])
    df["year_month"] = df[date_col].dt.to_period("M")
    monthly_avg = df.groupby("year_month")[num_cols].mean()

    print("\nMonthly Aggregated Data:")
    print(monthly_avg.head())
```

Monthly Aggregated Data:

	latitude	longitude	last_updated_epoch	\
year_month				
1970-01	2.607345e-17	2.343640e-17	1970-01-01	
	temperature_celsius	temperature_fahrenheit	wind_mph	\
year_month				
1970-01	-7.154268e-17	5.712146e-16	-6.872158e-17	
	wind_kph	wind_degree	pressure_mb	pressure_in ... \
year_month				...
1970-01	-9.432162e-17	-9.934550e-17	-7.228400e-15	2.172487e-15 ...
	air_quality_Carbon_Monoxide	air_quality_Ozone	\	
year_month				
1970-01	5.174459e-17	4.723191e-17		
	air_quality_Nitrogen_dioxide	air_quality_Sulphur_dioxide	\	
year_month				
1970-01	-1.269148e-16	3.097673e-17		
	air_quality_PM2.5	air_quality_PM10	air_quality_us-epa-index	\
year_month				
1970-01	-1.480831e-16	-3.146004e-17	2.862049e-19	
	air_quality_gb-defra-index	moon_illumination	wind_kmh	
year_month				
1970-01	8.952093e-17	8.412680e-17	1.044018e-18	

[1 rows x 31 columns]

Save Cleaned Data

```
In [47]: df.to_csv("GlobalWeatherRepository.csv", index=False)
```

```
In [48]: print("\nPreprocessing completed. Cleaned file saved.")
```

Preprocessing completed. Cleaned file saved.